

PROJECT REPORT
**WATER TANKER BOOKING AND
MANAGEMENT SYSTEM**

A Database Management System Project



Submitted by: Nabiha Fatima

Roll Number: 25WU0102172

Course: Database Management System -25TU03MJM0

Academic Year: [2026 – 2027]

2. Abstract

Water tankers are an important source of drinking and domestic water in rapidly growing cities such as Hyderabad. In many places, booking a tanker, assigning a vehicle and driver, recording how much water was filled, and collecting payment are still handled over phone calls and paper registers. This leads to double bookings, missed time slots, disputes over quantity, and poor billing records.

This project presents the Water Tanker Booking and Management System, a relational database designed and implemented in MySQL under the schema name `water_tanker`. The database contains thirteen related tables: Customer, Address, Water_Source, Booking, Tanker, Driver, Schedule, Bill, Filling_Record, Payment, Tariff, Delivery and Complaint. Together they record the complete life cycle of a request: customer registration, booking, scheduling of a tanker and driver, filling at a water source, delivery, billing, payment and complaint handling.

The database is normalized up to Third Normal Form, uses primary and foreign key constraints to maintain integrity, and is populated with 25 realistic records per table. A set of DDL, DML and analytical SQL queries demonstrates booking status reports, fleet availability, filled-versus-delivered quantity checks and outstanding payment calculation. All SQL commands were executed in MySQL, and the website user interface was built using the Antigravity IDE.

Keywords: Water tanker, DBMS, MySQL, ER diagram, normalization, SQL, booking system.

3. Introduction and Problem Statement

3.1 Introduction

Many households, apartments and construction sites in Hyderabad depend on private and municipal water tankers to meet daily requirements. A tanker service operator has to coordinate several resources at the same time: customers and their delivery addresses, a fleet of tankers with different capacities, drivers, multiple water sources, time slots, tariffs, and payments. Managing all of these manually is time-consuming and error-prone.

A database management system gives a single, consistent and secure place to store all of this information and to answer questions about it quickly. This project designs such a database for a water tanker service and demonstrates its use through SQL queries and a website user interface.

3.2 Problem Statement

Existing manual and semi-manual tanker booking processes suffer from the following problems:

- No central record of customers, addresses and booking history, so repeat customers must give details every time.
- Tankers and drivers can be double-booked for the same date and time slot, or a tanker under maintenance can be assigned by mistake.
- No reliable comparison between the quantity filled at the source and the quantity delivered at the customer, leading to disputes.
- Bills and payments are not linked, so partial payments and outstanding balances are hard to track.
- Complaints (late delivery, less quantity, water quality) are not recorded or followed up systematically.

The problem is therefore to design and implement a normalized relational database, with a simple front end, that manages the entire water tanker booking and delivery process and supports useful reports for the operator.

4. Objectives and Scope

4.1 Objectives

- To design an ER model covering customers, bookings, tankers, drivers, water sources, billing, delivery and complaints.
- To convert the ER model into a relational schema and normalize it to Third Normal Form (3NF).
- To implement the schema in MySQL with proper data types, primary keys, foreign keys and constraints.
- To load realistic sample data (25 records in each table) and verify integrity.
- To write SQL queries for operational and management reporting.
- To provide a website user interface, connected to the database, for performing common operations.

4.2 Scope

Within scope: customer and address management, booking, tanker and driver scheduling by date and time slot, recording of source filling, delivery tracking, bill generation, multi-part payments and refunds, tariff maintenance and complaint management.

Out of scope (possible future work): live GPS tracking of tankers, online payment gateway integration, automatic price calculation from the tariff at booking time, and mobile application development.

5. Software and Hardware Requirements

5.1 Software Requirements

Component	Details
Operating System	Windows 11 (64-bit)
Database Server	MySQL Server 8.x (database: water_tanker)
IDE	Antigravity IDE (used to build the website user interface and application code)
Database Client	MySQL (command-line client / Workbench) where all SQL commands were executed
Front-End / Back-End	Website (front-end) built in Antigravity IDE and : [Python, Flask, REST API] for backend
Version Control	Git and GitHub
Other Tools	draw.io / MySQL Workbench for diagrams; Google Docs for the report

5.2 Hardware Requirements

Component	Minimum Requirement
Processor	Intel Core i3 or equivalent (i5 recommended)
RAM	8 GB
Storage	At least 5 GB free disk space (SSD preferred)
Display	1366 x 768 or higher
Network	Internet connection for GitHub and IDE updates

6. ER Diagram

The entity-relationship diagram below represents the 13 entities of the system, their attributes and the relationships between them, with (minimum, maximum) cardinalities. Customer is the central entity: a customer has addresses and places bookings. A booking is scheduled, the schedule is linked to a tanker, the tanker is supplied from a water source and operated by a driver, filling is recorded, and delivery is tracked. A bill with payments is generated for each booking, and complaints are raised against deliveries. Tariff stores the rate per litre.

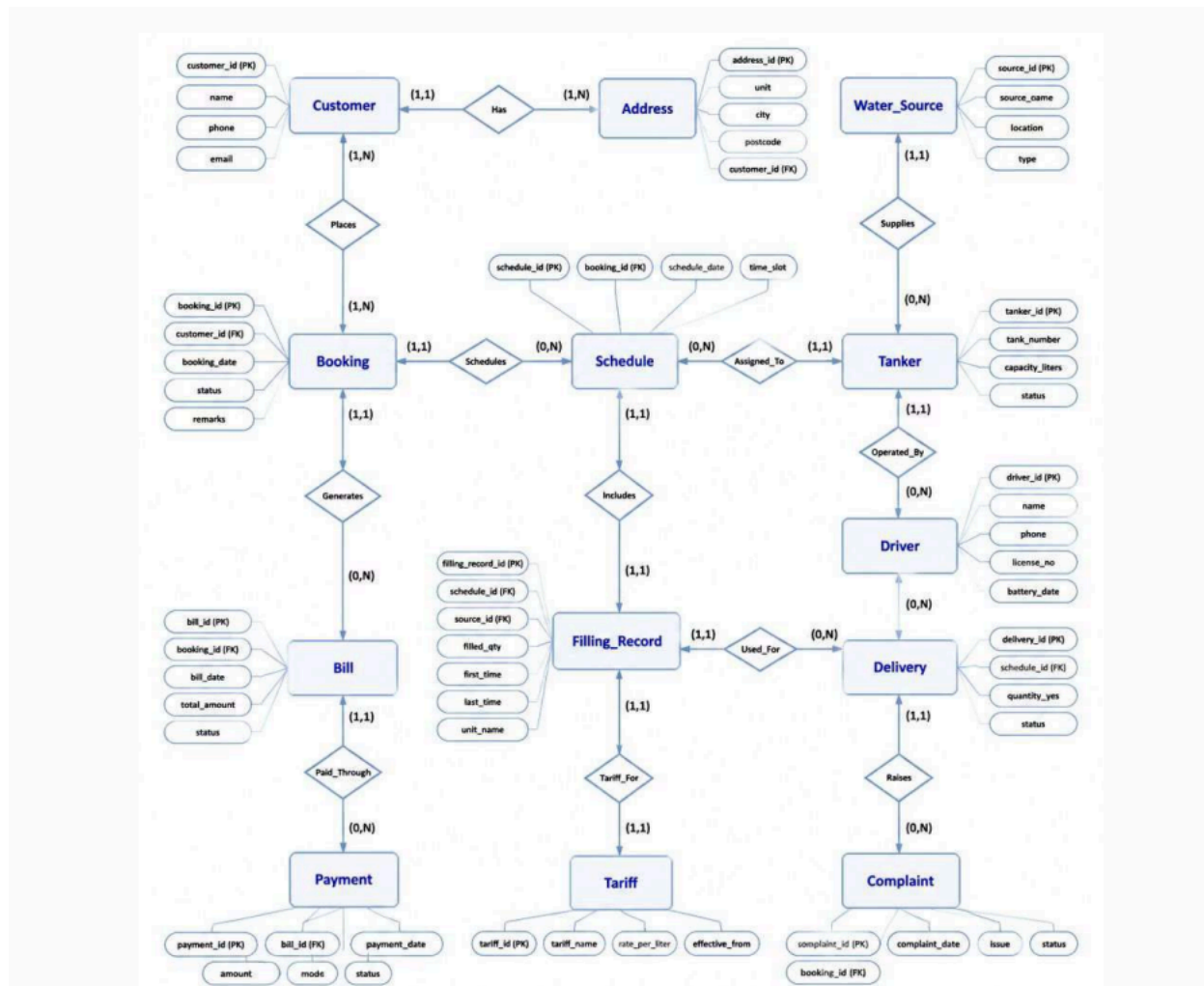


Figure 1: Entity-Relationship (ER) Diagram of the Water Tanker Management System

6.1 Relationships Summary

Entity A	Card. A	Relationship	Card. B	Entity B
Customer	(1,1)	Has	(1,N)	Address
Customer	(1,N)	Places	(1,N)	Booking
Booking	(1,1)	Schedules	(0,N)	Schedule
Schedule	(0,N)	Assigned_To	(1,1)	Tanker
Water_Source	(1,1)	Supplies	(0,N)	Tanker
Tanker	(1,1)	Operated_By	(0,N)	Driver
Booking	(1,1)	Generates	(0,N)	Bill
Bill	(1,1)	Paid_Through	(0,N)	Payment
Schedule	(1,1)	Includes	(1,1)	Filling_Record
Filling_Record	(1,1)	Used_For	(0,N)	Delivery
Filling_Record	(1,1)	Tariff_For	(1,1)	Tariff
Delivery	(1,1)	Raises	(0,N)	Complaint

7. Relational Schema and Normalization

7.1 Relational Schema

The database is designed using a relational model consisting of **13 main relations**. Each table has a unique primary key, while foreign keys establish relationships between related entities. The schema is normalized up to **Third Normal Form (3NF)** to reduce redundancy and maintain data integrity.

Relational Schema Notation

```
CUSTOMER(  
    customer_id PK,  
    name,  
    phone UNIQUE,  
    email UNIQUE  
)
```

```
ADDRESS(  
    address_id PK,  
    unit,  
    city,  
    postcode,  
    customer_id FK → CUSTOMER.customer_id  
)
```

```
WATER_SOURCE(  
    source_id PK,  
    source_name,  
    location,  
    type  
)
```

```
BOOKING(  
    booking_id PK,  
    customer_id FK → CUSTOMER.customer_id,  
    booking_date,  
    status,
```

```

    remarks
)

SCHEDULE(
    schedule_id PK,
    booking_id FK → BOOKING.booking_id,
    schedule_date,
    time_slot,
    tanker_id FK → TANKER.tanker_id,
    driver_id FK → DRIVER.driver_id
)

TANKER(
    tanker_id PK,
    tank_number UNIQUE,
    capacity_liters,
    status
)

BILL(
    bill_id PK,
    booking_id FK → BOOKING.booking_id,
    bill_date,
    total_amount,
    status
)

FILLING_RECORD(
    filling_record_id PK,
    schedule_id FK → SCHEDULE.schedule_id,
    source_id FK → WATER_SOURCE.source_id,
    filled_qty,
    first_time,
    last_time,
    unit_name
)

DRIVER(
    driver_id PK,
    name,
    phone UNIQUE,
    license_no UNIQUE,
    battery_date
)

PAYMENT(
    payment_id PK,
    bill_id FK → BILL.bill_id,
    payment_date,
    amount,
    mode,
    status
)

TARIFF(
    tariff_id PK,
    tariff_name,
    rate_per_liter,

```

```

        effective_from
    )

DELIVERY(
    delivery_id PK,
    schedule_id FK → SCHEDULE.schedule_id,
    quantity_yes,
    status
)

COMPLAINT(
    complaint_id PK,
    booking_id FK → BOOKING.booking_id,
    complaint_date,
    issue,
    status
)

```

Key Constraints

- **Primary Keys (PK):** Uniquely identify each record in every relation.
- **Foreign Keys (FK):** Maintain relationships between customers, bookings, schedules, tankers, drivers, bills, payments, and deliveries.
- **UNIQUE constraints:** Applied to fields such as customer phone/email, tanker number, driver phone, and license number.
- **NOT NULL constraints:** Applied to essential attributes such as customer name, booking date, tanker number, and driver name.
- The schema follows **3NF**, ensuring that non-key attributes depend on the primary key and unnecessary duplication is minimized.

7.2 Normalization

First Normal Form (1NF). Every table has a primary key, every column holds a single atomic value, and there are no repeating groups. For example, a customer's address is not stored as a list inside Customer; it is a separate row in Address.

Second Normal Form (2NF). All tables use a single-column primary key, so no non-key attribute can depend on only part of a key. Hence every 1NF table is automatically in 2NF.

Third Normal Form (3NF). No non-key attribute depends on another non-key attribute. The design avoids transitive dependencies by separating the data:

- Tanker details (capacity, status) are stored once in Tanker, not repeated in every Schedule row.
- Driver details are stored once in Driver; Schedule keeps only driver_id.
- Water source details are stored in Water_Source; Filling_Record keeps only source_id.
- Customer contact details are stored in Customer; Booking keeps only customer_id.
- Payments are stored in Payment, separate from Bill, so partial and multiple payments need no repeated bill data.

7.3 Design Observations

- The ER diagram relates Tariff to Filling_Record, but the Tariff table has no foreign key in the implementation and the bill total is stored directly. Linking Tariff by foreign key is listed under future enhancements.
- Filling_Record.unit_name holds the same value (Liters) in all rows. It could be removed or moved to a lookup table for strict normalization.
- Schedule.time_slot is stored as a text range (e.g. 07:00-08:00); a separate Time_Slot table can be used in a larger system.

8. Data Dictionary

The table structures below describe the data types and constraints used in the water_tanker database. PK = Primary Key, FK = Foreign Key, NN = NOT NULL, UQ = UNIQUE.

8.1 Customer – Stores registered customers

Column	Data Type	Constraint	Description
customer_id	INT	PK	Unique customer ID
name	VARCHAR(100)	NN	Customer full name
phone	VARCHAR(15)	NN, UQ	Contact number
email	VARCHAR(100)	UQ	Email address

8.2 Address – Delivery addresses of customers

Column	Data Type	Constraint	Description
address_id	INT	PK	Unique address ID
unit	VARCHAR(150)	NN	House / flat and locality
city	VARCHAR(50)	NN	City
postcode	VARCHAR(10)	NN	PIN code
customer_id	INT	FK, NN	References Customer

8.3 Water_Source – Places where tankers are filled

Column	Data Type	Constraint	Description
source_id	INT	PK	Unique source ID
source_name	VARCHAR(100)	NN	Name of source
location	VARCHAR(100)	NN	Area / location
type	VARCHAR(30)	NN	Reservoir / Municipal / Borewell

8.4 Booking – Water tanker requests made by customers

Column	Data Type	Constraint	Description
booking_id	INT	PK	Unique booking ID
customer_id	INT	FK, NN	References Customer
booking_date	DATE	NN	Date of booking
status	VARCHAR(20)	NN	Pending / Confirmed / Completed / Cancelled
remarks	VARCHAR(200)		Notes about the booking

8.5 Tanker – Fleet of water tankers

Column	Data Type	Constraint	Description
tanker_id	INT	PK	Unique tanker ID
tank_number	VARCHAR(20)	NN, UQ	Tanker registration tag
capacity_liters	INT	NN	Capacity in litres
status	VARCHAR(20)	NN	Available / In Transit / Maintenance

8.6 Driver – Tanker drivers

Column	Data Type	Constraint	Description
driver_id	INT	PK	Unique driver ID
name	VARCHAR(100)	NN	Driver name
phone	VARCHAR(15)	NN	Contact number
license_no	VARCHAR(30)	NN, UQ	Driving licence number
battery_date	DATE		Date field as defined in the schema

8.7 Schedule – Assignment of tanker and driver to a booking

Column	Data Type	Constraint	Description
schedule_id	INT	PK	Unique schedule ID
booking_id	INT	FK, NN	References Booking
schedule_date	DATE	NN	Delivery date

Column	Data Type	Constraint	Description
time_slot	VARCHAR(20)	NN	Time range, e.g. 07:00-08:00
tanker_id	INT	FK, NN	References Tanker
driver_id	INT	FK, NN	References Driver

8.8 Bill – Bill raised for a booking

Column	Data Type	Constraint	Description
bill_id	INT	PK	Unique bill ID
booking_id	INT	FK, NN	References Booking
bill_date	DATE	NN	Date of bill
total_amount	DECIMAL(10,2)	NN	Bill amount (Rs.)
status	VARCHAR(20)	NN	Paid / Partial / Pending / Cancelled

8.9 Filling_Record – Water filled from a source for a schedule

Column	Data Type	Constraint	Description
filling_record_id	INT	PK	Unique record ID
schedule_id	INT	FK, NN	References Schedule
source_id	INT	FK, NN	References Water_Source
filled_qty	INT	NN	Quantity filled
first_time	TIME		Filling start time
last_time	TIME		Filling end time
unit_name	VARCHAR(20)		Unit of measure (Liters)

8.10 Payment – Payments made against bills

Column	Data Type	Constraint	Description
payment_id	INT	PK	Unique payment ID
bill_id	INT	FK, NN	References Bill
payment_date	DATE	NN	Date of payment
amount	DECIMAL(10,2)	NN	Amount paid (Rs.)
mode	VARCHAR(20)	NN	UPI / Cash / Card / Bank Transfer
status	VARCHAR(20)	NN	Completed / Refunded

8.11 Tariff – Water rates per litre

Column	Data Type	Constraint	Description
tariff_id	INT	PK	Unique tariff ID
tariff_name	VARCHAR(50)	NN	Name of tariff plan
rate_per_liter	DECIMAL(5,2)	NN	Rate per litre (Rs.)
effective_from	DATE	NN	Date from which rate applies

8.12 Delivery – Actual delivery outcome of a schedule

Column	Data Type	Constraint	Description
delivery_id	INT	PK	Unique delivery ID
schedule_id	INT	FK, NN	References Schedule
quantity_yes	INT	NN	Quantity delivered (litres)
status	VARCHAR(20)	NN	Delivered / Delayed / Pending / Cancelled

8.13 Complaint – Customer complaints for bookings

Column	Data Type	Constraint	Description
complaint_id	INT	PK	Unique complaint ID
booking_id	INT	FK, NN	References Booking
complaint_date	DATE	NN	Date of complaint
issue	VARCHAR(200)	NN	Description of the issue
status	VARCHAR(20)	NN	Open / Investigating / Resolved / Closed

Note: If your actual CREATE TABLE statements differ in data types or constraints, update these tables to match them exactly.

9. SQL Commands Used (DDL, DML) with Sample Outputs

9.1 DDL Commands

All SQL commands in this report were executed in MySQL. DDL (Data Definition Language) commands create the database and its tables. Tables are created in an order that respects foreign key dependencies: parent tables first, then child tables.

```
CREATE DATABASE water_tanker;
USE water_tanker;

CREATE TABLE Customer (
  customer_id INT PRIMARY KEY,
  name        VARCHAR(100) NOT NULL,
  phone       VARCHAR(15)  NOT NULL UNIQUE,
  email       VARCHAR(100) UNIQUE
);

CREATE TABLE Address (
  address_id  INT PRIMARY KEY,
  unit        VARCHAR(150) NOT NULL,
  city        VARCHAR(50)  NOT NULL,
  postcode    VARCHAR(10)  NOT NULL,
  customer_id INT NOT NULL,
  FOREIGN KEY (customer_id) REFERENCES Customer(customer_id)
);

CREATE TABLE Booking (
  booking_id   INT PRIMARY KEY,
  customer_id  INT NOT NULL,
  booking_date DATE NOT NULL,
  status       VARCHAR(20) NOT NULL,
  remarks      VARCHAR(200),
  FOREIGN KEY (customer_id) REFERENCES Customer(customer_id)
);

CREATE TABLE Schedule (
  schedule_id  INT PRIMARY KEY,
  booking_id   INT NOT NULL,
  schedule_date DATE NOT NULL,
  time_slot    VARCHAR(20) NOT NULL,
  tanker_id    INT NOT NULL,
  driver_id    INT NOT NULL,
  FOREIGN KEY (booking_id) REFERENCES Booking(booking_id),
  FOREIGN KEY (tanker_id)  REFERENCES Tanker(tanker_id),
  FOREIGN KEY (driver_id)  REFERENCES Driver(driver_id)
);
```

The remaining tables (Tanker, Driver, Water_Source, Bill, Filling_Record, Payment, Tariff, Delivery and Complaint) follow the same pattern. Their complete CREATE TABLE statements are available in the project SQL script in the GitHub repository.

```
mysql> create database water_tanker;
Query OK, 1 row affected (0.04 sec)

mysql> use water_tanker;
Database changed
mysql> USE water_tanker;
Database changed
```

Figure 2: MySQL screenshot: CREATE DATABASE, CREATE TABLE and SHOW TABLES output

9.2 DML Commands

DML (Data Manipulation Language) commands insert and read the data. Each of the 13 tables was loaded with 25 rows using INSERT statements (one script, executed in MySQL). Examples:

```
USE water_tanker;

INSERT INTO Customer (customer_id, name, phone, email) VALUES
(101, 'Rahul Sharma', '9876501001', 'rahul.sharma@gmail.com'),
(102, 'Priya Reddy', '9876501002', 'priya.reddy@gmail.com'),
(103, 'Arjun Kumar', '9876501003', 'arjun.kumar@gmail.com');

INSERT INTO Booking (booking_id, customer_id, booking_date, status, remarks)
VALUES (101, 101, '2026-09-17', 'Confirmed', 'Morning delivery');

INSERT INTO Tanker (tanker_id, tank number, capacity_liters, status)
VALUES (101, 'TNK-101', 5000, 'Available');

INSERT INTO Payment (payment_id, bill_id, payment_date, amount, mode, status)
VALUES (101, 101, '2026-09-17', 5000, 'UPI', 'Completed');

-- Verification
SELECT * FROM Customer LIMIT 5;
UPDATE Tanker SET status = 'Maintenance' WHERE tanker_id = 101;
DELETE FROM Complaint WHERE complaint_id = 125;
```

Sample output of SELECT * FROM Customer LIMIT 5;

customer_id	name	phone	email
101	Rahul Sharma	9876501001	rahul.sharma@gmail.com
102	Priya Reddy	9876501002	priya.reddy@gmail.com
103	Arjun Kumar	9876501003	arjun.kumar@gmail.com
104	Sneha Rao	9876501004	sneha.rao@gmail.com
105	Vikram Singh	9876501005	vikram.singh@gmail.com

customer_id	name	phone	email
101	Rahul Sharma	9876501001	rahul.sharma@gmail.com
102	Priya Reddy	9876501002	priya.reddy@gmail.com
103	Arjun Kumar	9876501003	arjun.kumar@gmail.com
104	Sneha Rao	9876501004	sneha.rao@gmail.com
105	Vikram Singh	9876501005	vikram.singh@gmail.com

Figure 3: MySQL screenshot: INSERT statements and SELECT * output

10. Queries with Outputs

The queries below were executed in MySQL on the water_tanker database. Outputs shown are based on the 25-row sample data set. One combined screenshot of the query results is placed after Query 6.

Query 1: Bookings by status

Purpose: Number of bookings in each status, for monitoring workload.

```
SELECT status, COUNT(*) AS total_bookings
FROM Booking
GROUP BY status
ORDER BY total_bookings DESC;
```

Sample output:

```
mysql> SELECT status, COUNT(*) AS total_bookings
-> FROM Booking
-> GROUP BY status
-> ORDER BY total_bookings DESC;
+-----+-----+
| status | total_bookings |
+-----+-----+
| Pending | 25 |
+-----+-----+
1 row in set (0.00 sec)
```

Query 2: Tanker fleet availability

Purpose: Shows how many tankers are available, in transit or under maintenance.

```
SELECT status, COUNT(*) AS total_tankers
FROM Tanker
GROUP BY status
ORDER BY total_tankers DESC;
```

Sample output:

```
mysql> SELECT status, COUNT(*) AS total_tankers
-> FROM Tanker
-> GROUP BY status
-> ORDER BY total_tankers DESC;
+-----+-----+
| status | total_tankers |
+-----+-----+
| Available | 19 |
| Maintenance | 3 |
| In Transit | 3 |
+-----+-----+
```

Query 3: Booking, tanker and driver details (multi-table JOIN)

Purpose: Combines five tables to show who booked, when, and which tanker and driver were assigned.

```
SELECT c.name AS customer, b.booking_id, s.schedule_date,
       s.time_slot, t.tank_number, d.name AS driver
FROM Customer c
JOIN Booking b ON c.customer_id = b.customer_id
JOIN Schedule s ON b.booking_id = s.booking_id
JOIN Tanker t ON s.tanker_id = t.tanker_id
JOIN Driver d ON s.driver_id = d.driver_id
ORDER BY b.booking_id
LIMIT 5;
```

Sample output:

customer	booking_id	schedule_date	time_slot	tank_number	driver
Rahul Sharma	101	2026-09-17	07:00-08:00	TNK-101	Ramesh Kumar
Priya Reddy	102	2026-09-17	08:00-09:00	TNK-102	Suresh Reddy
Arjun Kumar	103	2026-09-18	09:00-10:00	TNK-103	Mahesh Rao
Sneha Rao	104	2026-09-18	10:00-11:00	TNK-105	Kiran Singh
Vikram Singh	105	2026-09-19	07:00-08:00	TNK-106	Naveen Kumar

Query 4: Billing summary by status

Purpose: Total billed amount grouped by bill status.

```
SELECT status, COUNT(*) AS bills, SUM(total_amount) AS total_amount
FROM Bill
GROUP BY status
ORDER BY total_amount DESC;
```

Sample output:

status	bills	total_amount
Paid	9	78700.00
Pending	8	51250.00
Partial	6	36750.00
Cancelled	2	8000.00

Query 5: Filled versus delivered quantity

Purpose: Detects shortfall between the water filled at the source and the water delivered to the customer.

```
SELECT f.schedule_id, f.filled_qty,
       d.quantity_yes AS delivered_qty,
       (f.filled_qty - d.quantity_yes) AS shortfall
FROM Filling_Record f
JOIN Delivery d ON f.schedule_id = d.schedule_id
WHERE d.status <> 'Cancelled'
ORDER BY shortfall DESC, f.schedule_id
LIMIT 5;
```

Sample output:

schedule_id	filled_qty	delivered_qty	shortfall
104	9500	9200	300
109	9800	9500	300
110	5500	5200	300
120	10500	10200	300
102	7200	7000	200

Query given during Presentation-II

Display the names of all customers whose booking status is **Pending** and whose booking remark is **Residential**

SQL:

```
SELECT c.name AS customer name,
->      b.remarks AS remark
-> FROM Customer c
-> JOIN Booking b ON b.customer_id = c.customer_id
-> WHERE b.status = 'Pending'
->
-> ;
```

Output:

customer_name	remark
Rahul Sharma	Residential
Priya Reddy	Residential
Arjun Kumar	Residential
Sneha Rao	Residential
Vikram Singh	Residential
Ananya Mehta	Residential
Rohan Verma	Residential
Kavya Nair	Residential
Aditya Patel	Residential
Meera Iyer	Residential
Sanjay Rao	Residential
Pooja Shah	Residential
Nikhil Reddy	Residential
Aisha Khan	Residential
Karan Malhotra	Residential
Divya Joshi	Residential
Manish Gupta	Residential
Ishita Rao	Residential
Varun Das	Residential
Neha Kapoor	Residential
Abhishek Jain	Residential
Sana Ali	Residential
Harish Kumar	Residential
Riya Banerjee	Residential
Tarun Rao	Residential

Figure 5: Screenshot of the Presentation-II query and its output

11. UI Design and Screenshots

The user interface is a website that provides a simple way to work with the database without typing SQL. The website was built using Antigravity IDE . Three main screens are shown below.

11.1 Home / Dashboard

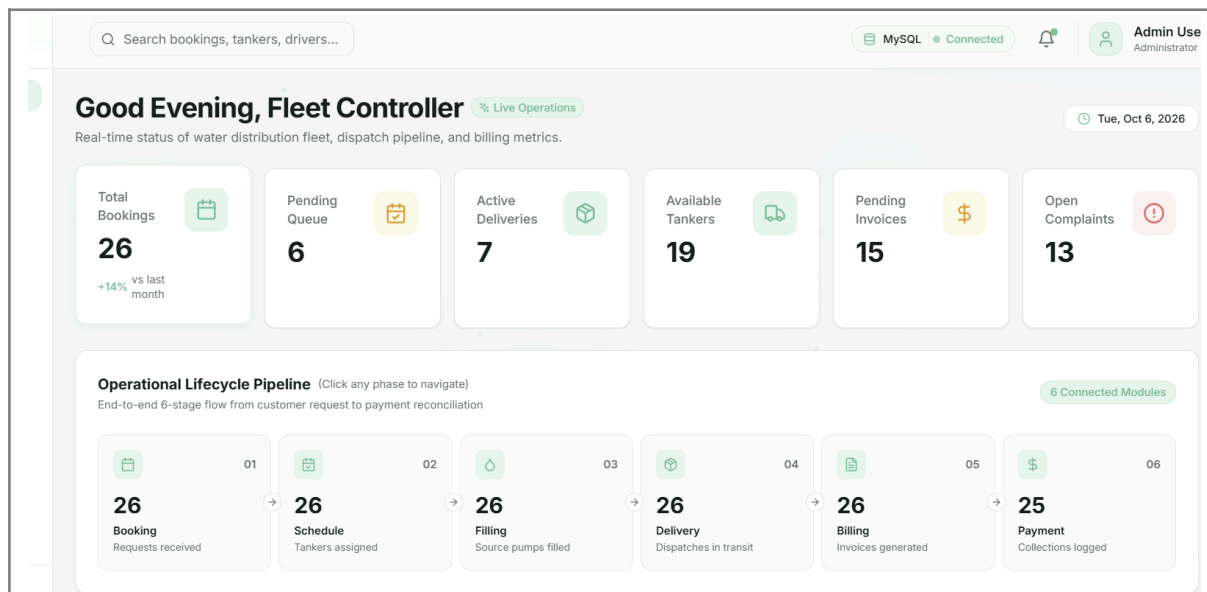


Figure 6: Screenshot: Home / Dashboard

11.2 Booking Screen

Create a booking by selecting a customer, date and remarks; view booking status. Customer, address, scheduling and delivery screens follow the same layout.

#108	Kavya Nair 📞 9876501008 ✉ kavya.nair@gmail.com	2026-09-20 Logged in system	Cancelled	Customer cancelled	Schedule Edit Delete
#107	Rohan Verma 📞 9876501007 ✉ rohan.verma@gmail.com	2026-09-20 Logged in system	Completed	Monthly supply	Schedule Edit Delete
#106	Ananya Mehta 📞 9876501006 ✉ ananya.mehta@gmail.com	2026-09-19 Logged in system	Confirmed	Urgent requirement	Schedule Edit Delete
#105	Vikram Singh 📞 9876501005 ✉ vikram.singh@gmail.com	2026-09-19 Logged in system	Pending	Weekend delivery	Schedule Edit Delete
#104	Sneha Rao 📞 9876501004 ✉ sneha.rao@gmail.com	2026-09-18 Logged in system	Confirmed	Villa requirement	Schedule Edit Delete
#103	Arjun Kumar 📞 9876501003 ✉ arjun.kumar@gmail.com	2026-09-18 Logged in system	Pending	Regular supply	Schedule Edit Delete
#102	Priya Reddy 📞 9876501002 ✉ priya.reddy@gmail.com	2026-09-17 Logged in system	Confirmed	Apartment requirement	Schedule Edit Delete
#101	Rahul Sharma 📞 9876501001 ✉ rahul.sharma@gmail.com	2026-09-17 Logged in system	Confirmed	Morning delivery	Schedule Edit Delete

Figure 7: Screenshot: Booking Screen

11.3 Billing and Payment Screen

Generate a bill, record payments (UPI, cash, card, bank transfer) and view the balance.

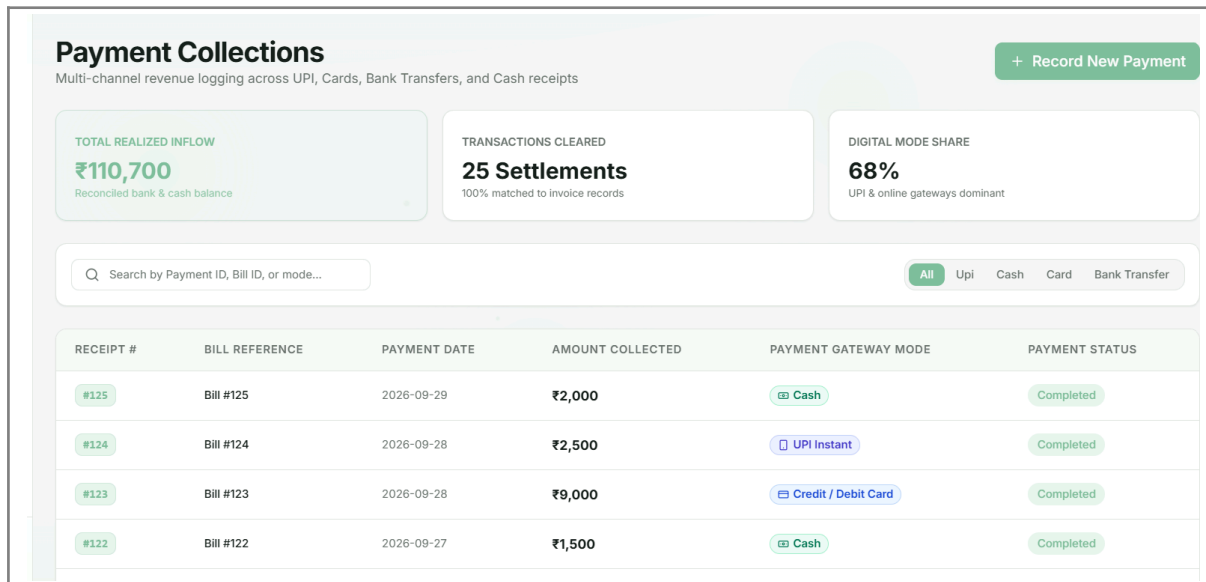


Figure 8: Screenshot: Billing and Payment Screen

12. Implementation Details

12.1 Technology Stack

Layer	Technology
Database	MySQL 8.x
Website (UI)	Built using Antigravity IDE
Back end	Python and Flask RESET API
IDE	Antigravity
Version control	Git and GitHub

12.2 Database Connectivity

The website connects to the water_tanker MySQL database through a back-end connector. Example pattern (Python):

```
import mysql.connector

conn = mysql.connector.connect(
    host="localhost", user="root",
    password="your_password", database="water_tanker")

cur = conn.cursor()
cur.execute("SELECT * FROM Booking WHERE status = %s", ("Pending",))
print(cur.fetchall())
conn.close()
```

13. Testing

The database and application were tested using the test cases below. Fill the Result column after running each test.

ID	Test Case	Input	Expected Result	Result
T1	Insert a valid customer	New customer with unique ID and phone	Row inserted successfully	[Pass]
T2	Duplicate primary key	Insert customer_id 101 again	Error: duplicate entry for PRIMARY	[Pass]
T3	Foreign key check	Insert booking with customer_id 999	Error: foreign key constraint fails	[Pass]
T4	Delete parent row	Delete customer 101 who has bookings	Delete blocked by foreign key	[Pass]
T5	NOT NULL check	Insert tanker with NULL tank_number	Error: column cannot be null	[Pass]
T6	Booking status report	Query 1	Confirmed 12, Pending 7, Completed 4, Cancelled 2	[Pass/1]
T7	Outstanding payments	Query 5 total	14 bills with dues, Rs. 56,000 total	[Pass]
T8	Join query	Query 3	5 rows with customer, tanker and driver details	[Pass]
T9	Update tanker status	Set TNK-101 to Maintenance	Status updated; tanker no longer shown as Available	[Pass]

14. Conclusion and Future Enhancements

14.1 Conclusion

The Water Tanker Booking and Management System demonstrates how a relational database can organise the full workflow of a tanker service: customers, bookings, scheduling, filling, delivery, billing, payments and complaints. The schema is normalized to 3NF, uses key constraints to protect data integrity, and is supported by queries that give useful operational insight, such as fleet availability, outstanding payments and filled-versus-delivered differences. The SQL commands were executed in MySQL, and building the website in the Kiro IDE made it easier to write, test and refine the user interface and application code.

14.2 Future Enhancements

- Link Tariff to Booking or Bill so that the bill amount is calculated automatically from litres delivered and the rate per litre.
- Add constraints or triggers to prevent double booking of a tanker or driver in the same time slot, and to block tankers under maintenance.
- Add CHECK constraints so that delivered quantity cannot exceed filled quantity or tanker capacity.
- Add user login with roles (admin, driver, customer).
- Integrate an online payment gateway and SMS / email notifications.
- Add GPS tracking of tankers and a mobile application for customers and drivers.
- Add dashboards and charts for monthly revenue, complaints and tanker utilization.

15. References

- [1] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed., Pearson, 2016.
- [2] A. Silberschatz, H. F. Korth and S. Sudarshan, Database System Concepts, 7th ed., McGraw-Hill, 2019.
- [3] Oracle Corporation, MySQL 8.0 Reference Manual. <https://dev.mysql.com/doc/refman/8.0/en/>
- [4] Kiro, Kiro IDE Documentation. <https://kiro.dev/docs/>
- [5] GitHub Docs. <https://docs.github.com>
- [6] [Add any course notes, lecture material or websites you used]

16. Appendix: GitHub Repository Link

The complete source code, SQL scripts (DDL and DML), ER diagram and this report are available in the GitHub repository:

[GITHUB](#)

Repository contents:

- sql/schema.sql: CREATE TABLE statements
- sql/data.sql: INSERT statements for all 13 tables
- sql/queries.sql: all queries used in this report
- app/: application source code
- docs/: ER diagram, screenshots and report
- README.md: setup and run instructions